

Experimental search for gap sequences for Shellsort

Current solution and first results

Thesis / Speaker: inż. Adam Cherek
Supervisor: dr hab. inż. Paweł Czarnul



WYDZIAŁ ELEKTRONIKI,
TELEKOMUNIKACJI
I INFORMATYKI

Searching pipeline

In the current version of the searching pipeline, each run:

1. Searches for a certain sorting range (input array length)
2. Starts with: 4 known sequences (Tokuda, Ciura, Lee, SEJ) + 96 randomized sequences
3. Runs endless iterations in which:
 - a. All sequences are tested on 100 random arrays to be sorted (same for each algorithm in the iteration),
 - b. Results are used to evaluate each sequence with a fitness function, and then they are ranked based on average score,
 - c. The best sequence is compared again this time only with Ciura's and SEJ sequences, and added to the list of candidate sequences if it wins,
 - d. Selected population-based algorithm creates a new population for next iteration.
4. Ends manually after a few hours, when the number of found sequences either satisfies researcher or is not promising for maintaining the procedure.

First Algorithm – GA v1

Genetic Algorithm version 1:

- Top 50% of previous population is crossed and creates 50% of new population – 2 parent sequences switch half of their sequence with each other, creating 2 children.
- Remaining 50% is filled with randomized sequences.
- No survivors, no mutation.

This approach worked well on small data, generating both good number of sequences and good variety between them.

It was quickly verified to be ineffective for bigger sorting ranges.

Second Algorithm – GA v2

Genetic Algorithm version 2:

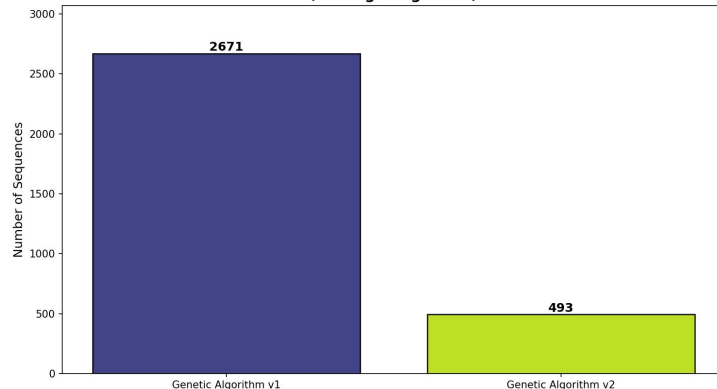
- Top 10% of previous population is kept as survivors.
- Top 50% of previous population is crossed and creates 50% of new population – 2 parent sequences switches half of their sequence with each other, creating 2 children.
- Each sequence in this new 60% has a 10% chance of being mutated.
- Remaining 40% is filled with randomized sequences.

This approach was more reliable, working for all tested sorting ranges and generating even bigger number of candidate sequences.

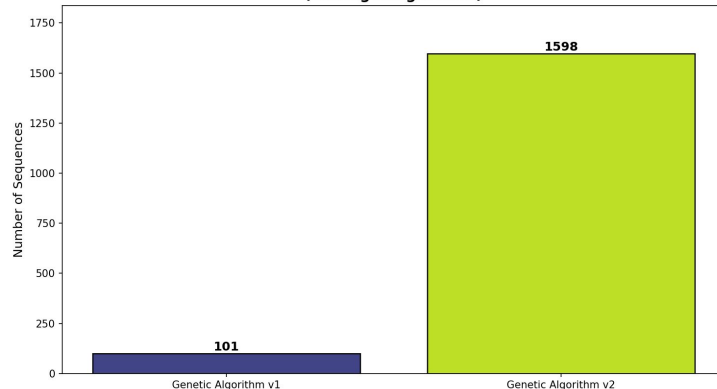
It turned out to quickly get stuck in a local optimum, often generating sequences only slightly different than already established sequences (like Ciura or SEJ).

Comparison of GA v1 and v2

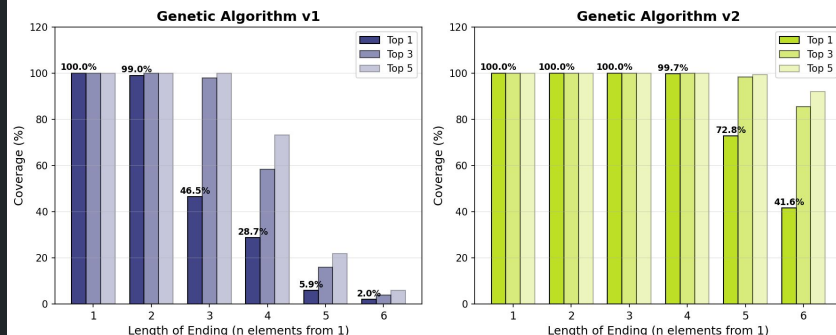
Number of Gap Sequences Generated by Algorithm
(Sorting Range: 500)



Number of Gap Sequences Generated by Algorithm
(Sorting Range: 2000)



Coverage of Top Sequence Endings (from 1) by Algorithm
(Sorting Range: 2000)



Critical flaw of GA v2:

The algorithm is getting stuck in local optimum (often around one of known sequences like Ciura or SEJ).

Generates many candidate sequences but with low research value as they are mostly the same sequences with one gap slightly changed – around 95% found via mutation.

Genetic Algorithm v2 iteration 44785:

```
Gaps sequences:
44785|Survivor|1: 1477 343 145 62 27 10 4 1
44785|Survivor|2: 1477 343 145 62 27 10 4 1
44785|Survivor|3: 1477 343 145 62 27 10 4 1
44785|Survivor|4: 1477 343 145 62 27 10 4 1
44785|Survivor|5: 1477 343 145 62 27 10 4 1
44785|Survivor|6: 1477 343 145 62 27 10 4 1
44785|Survivor|7|Mutated: 1477 343 145 62 27 10 3 1
44785|Survivor|8: 1477 343 145 62 27 10 4 1
44785|Survivor|9: 1477 343 145 62 27 10 4 1
44785|Survivor|10: 1477 343 145 62 27 10 4 1
44785|Child|1|Mutated: 1345 379 145 62 27 10 4 1
44785|Child|2: 1477 343 145 62 27 10 4 1
44785|Child|3: 1477 343 145 62 27 10 4 1
44785|Child|4: 1477 343 145 62 27 10 4 1
44785|Child|5: 1477 343 145 62 27 10 4 1
44785|Child|6: 1477 343 145 62 27 10 4 1
44785|Child|7: 1477 343 145 62 27 10 4 1
44785|Child|8: 1477 343 145 62 27 10 4 1
44785|Child|9: 1477 343 145 62 27 10 4 1
44785|Child|10: 1477 343 145 62 27 10 4 1
44785|Child|11: 1477 343 145 62 27 10 4 1
44785|Child|12: 1477 343 145 62 27 10 4 1
44785|Child|13: 1477 343 145 62 27 10 4 1
44785|Child|14: 1477 343 145 62 27 10 4 1
44785|Child|15: 1477 343 145 62 27 10 4 1
44785|Child|16|Mutated: 1477 343 145 62 26 9 4 1
44785|Child|17: 1477 343 145 62 27 10 4 1
44785|Child|18|Mutated: 1477 343 145 62 27 10 3 1
44785|Child|19: 1477 343 145 62 27 10 4 1
44785|Child|20: 1477 343 145 62 27 10 4 1
44785|Child|21: 1477 343 145 62 27 10 4 1
44785|Child|22: 1477 343 145 62 27 10 4 1
44785|Child|23: 1477 343 145 62 27 10 4 1
44785|Child|24: 1477 343 145 62 27 10 4 1
44785|Child|25: 1477 343 145 62 27 10 4 1
```

Looking for Inspiration

At this point I decided to check other algorithms that could be well-suited for this challenge. The choice fell on two swarm-intelligence algorithms:

Cuckoo Search (CS):

Algorithm inspired by aggressive reproduction strategy of cuckoo species. Core idea is driven by Lévy Flight – random walk with heavy-tailed distribution.

- + Lévy Flights provide balanced approach on mutation, where many small changes are paired with occasional long jumps.
- Very dependent on the used hyper-parameters (beta and step-length).
- Better for exploitation than exploration.

Artificial Bee Colony (ABC):

Algorithm inspired by intelligent foraging behaviour of honey bee swarm. Core idea is driven by division of roles into employed, onlooker and scout bees.

- + Distance based approach on finding neighbor solutions seem to be working well in this scenario both for exploration and exploitation .
- Requires additional evaluations during generation of new population – it's very costly for the given problem.
- More complex implementation, less room for custom modifications.

Third Algorithm – GA v3

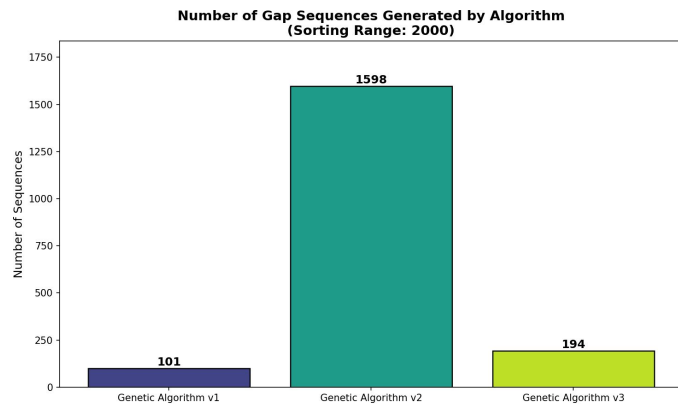
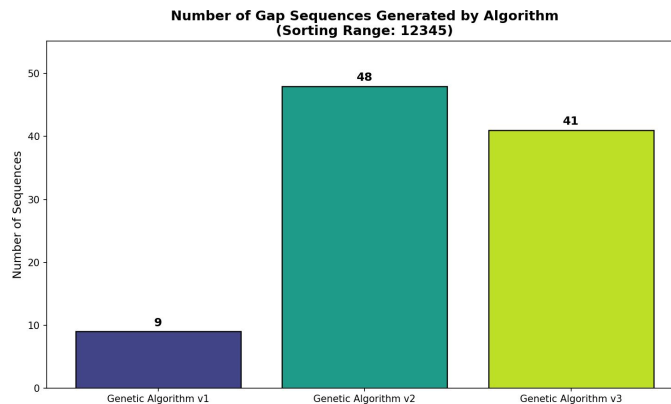
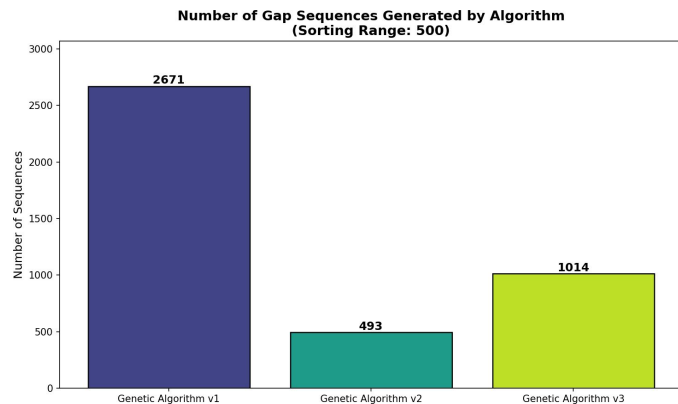
Genetic Algorithm version 3:

- Top 1 sequence of previous population is kept as survivor.
- Top 33% of previous population is crossed, averaged and treated with Lévy Flight to create 66% of new population.
- Each sequence in this new 66(+1)% has a 10% chance of being mutated.
- Remaining 33% is filled with randomized sequences.

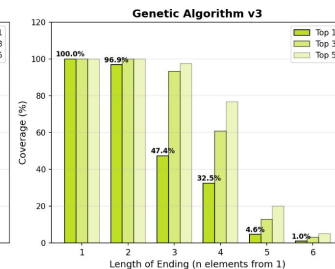
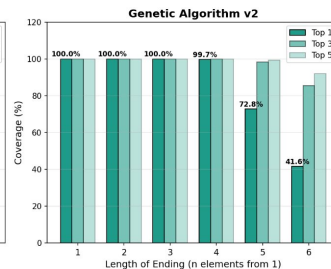
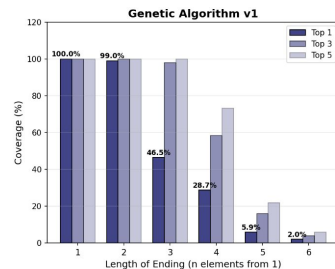
This is the currently used solution that has proven to be much more stable than previous approaches, providing a decent number of candidate gap sequences in all sorting ranges, and at the same time a good variety between them.

There is still room for improvement as methods recycled from CS and ABC seem to be a little underperforming in this implementation.

Comparison of GA v1, v2 and v3



Coverage of Top Sequence Endings (from 1) by Algorithm (Sorting Range: 2000)



Data Gathering

“GAv3 is the heart of our current solution, but the heart can’t effectively pump all the blood through one artery: it needs a capillary network to distribute the workload”

~ Adam Cherek 10.06.2025

Current solution is far from optimal but it works fast enough for sorting ranges up to 10000 (already much better than initial single-threaded prototype). Each population generated by genetic algorithm is evaluated by thousands of sorting procedures, running in parallel on all logical cores.

Such process often takes a few hours (most often whole night) to go through 50K - 4B iterations, producing on average 500-1000 unique candidate sequences, stored in text files for designated sorting ranges.

```
≡ CandidateGapSequences1000_GAv3.txt
6|Child_Cross|34|Mutated: 646 153 69 33 14 4 1
34|Survivor|1: 217 72 31 14 4 1
37|Child_Avg|43|Mutated: 203 70 31 11 4 1
38|Child_Avg|3: 209 76 27 14 4 1
55|Child_Cross|38: 214 77 28 15 4 1
56|Child_Levy|28: 246 101 30 13 5 1
58|Child_Cross|38: 214 77 28 13 4 1
68|Child_Levy|52: 279 72 30 13 5 1
81|Child_Levy|32: 268 71 27 13 5 1
84|Survivor|1|Mutated: 268 71 27 14 5 1
93|Child_Levy|24: 195 67 30 13 5 1
94|Child_Cross|13: 268 71 27 14 4 1
108|Child_Cross|10: 588 154 55 26 14 5 1
130|Child_Levy|64: 153 63 31 14 5 1
131|Child_Cross|21: 588 154 55 21 12 5 1
134|Child_Levy|16: 267 81 30 14 5 1
147|Child_Levy|52|Mutated: 179 74 25 11 5 1
150|Child_Cross|13: 179 74 25 11 4 1
156|Survivor|1: 163 60 28 11 4 1
158|Child_Levy|56: 341 74 35 12 5 1
166|Child_Levy|40: 279 95 35 13 5 1
```

Current Results

Is it really that simple?



Hundreds of Better Gap Sequences? – Case Study

Theoretically it's true, I've already found over 40 thousands candidate sequences for different sorting ranges, and all of them were somehow better than already established ones.

After quick analysis on candidate gap sequences found for sorting range 10000, it turned out that they not only beat named sequences in this range, but best of them can also perform better in other ranges for which they were not specialized – in this case range 8000.

So where is the catch?

```
8000 range / 10000 tryouts
Results:
GAv3-10000|Top2: 2304 919 390 159 69 28 12 5 1
| Time: 0.474339ms | Operations: 132946 | Wins: 2829

SEJ: 3313 1272 488 187 72 27 10 4 1
| Time: 0.50027ms | Operations: 134262 | Wins: 191

Ciura: 1750 701 301 132 57 23 10 4 1
| Time: 0.513553ms | Operations: 133922 | Wins: 530
```

```
10000 range / 10000 tryouts
Results:
GAv3-10000|Top1: 3170 983 432 191 93 35 13 5 1
| Time: 0.678871ms | Operations: 172995 | Wins: 3833

GAv3-10000|Top2: 2304 919 390 159 69 28 12 5 1
| Time: 0.672907ms | Operations: 173090 | Wins: 3357

GAv3-10000|Top3: 2905 891 355 160 75 29 11 5 1
| Time: 0.659833ms | Operations: 173319 | Wins: 2404

SEJ: 8627 3313 1272 488 187 72 27 10 4 1
| Time: 0.701247ms | Operations: 174711 | Wins: 231

Ciura: 1750 701 301 132 57 23 10 4 1
| Time: 0.682103ms | Operations: 175080 | Wins: 173
```

Challenges in Analysis

- Even with averaging thousands of tryouts, runtime remains very unstable as a method of comparison/evaluation.
- Simplified fitness function was used – number of operations performed by Shellsort with certain gap sequence.
- Gap sequences found this way are indeed winning in terms of operations count, but this is not equal to being faster.
- While the average number of operations remains proportional with each evaluation (meaning similar ranking every time), average runtime tends to differ by a few milliseconds both ways (leading to different ranking each run)

1000 range / 10000 tryouts – Results:

1. SEJ: 488 187 72 27 10 4 1
Time: 0.0421459ms | Operations: 11090.1 | Wins: 4275
2. GAv3-10000|Top1: 3170 983 432 191 93 35 13 5 1
Time: 0.0412062ms | Operations: 11116.3 | Wins: 3349
3. GAv3-10000|Top2: 2304 919 390 159 69 28 12 5 1
Time: 0.046213ms | Operations: 11186.3 | Wins: 1937
4. Tokuda: 525 233 103 46 20 9 4 1
Time: 0.0450164ms | Operations: 11358.2 | Wins: 91
5. Ciura: 701 301 132 57 23 10 4 1
Time: 0.0402952ms | Operations: 11396.1 | Wins: 121
6. Lee: 516 230 102 45 20 9 4 1
Time: 0.0447081ms | Operations: 11429.2 | Wins: 38
7. GAv3-10000|Top3: 2905 891 355 160 75 29 11 5 1
Time: 0.0422848ms | Operations: 11460.5 | Wins: 189

10000 range / 10000 tryouts – Results:

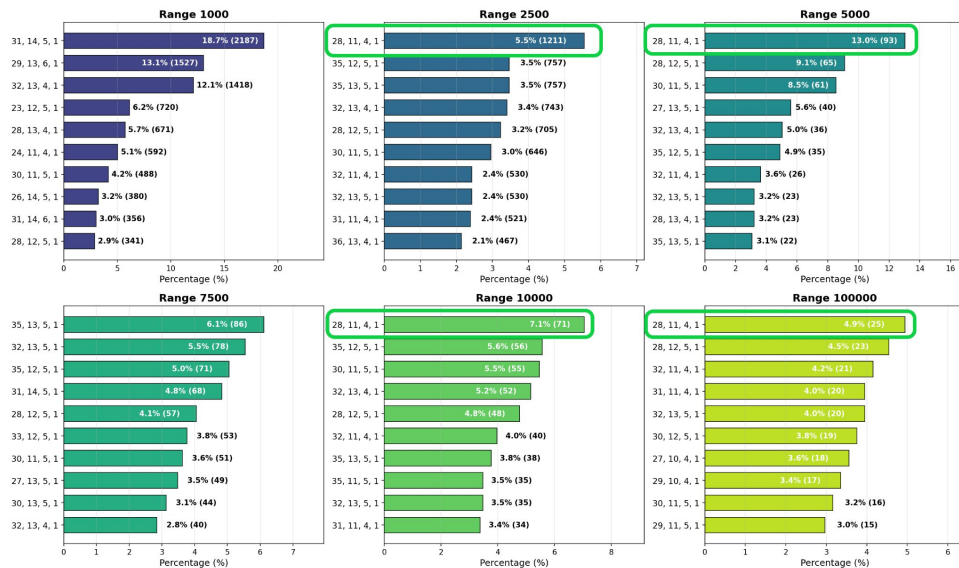
1. GAv3-10000|Top1: 3170 983 432 191 93 35 13 5 1
Time: 0.678871ms | Operations: 172995 | Wins: 3833
2. GAv3-10000|Top2: 2304 919 390 159 69 28 12 5 1
Time: 0.672907ms | Operations: 173090 | Wins: 3357
3. GAv3-10000|Top3: 2905 891 355 160 75 29 11 5 1
Time: 0.659833ms | Operations: 173319 | Wins: 2404
4. SEJ: 8627 3313 1272 488 187 72 27 10 4 1
Time: 0.701247ms | Operations: 174711 | Wins: 231
5. Ciura: 1750 701 301 132 57 23 10 4 1
Time: 0.682103ms | Operations: 175080 | Wins: 173
6. Tokuda: 5985 2660 1182 525 233 103 46 20 9 4 1
Time: 0.64616ms | Operations: 176336 | Wins: 0
7. Lee: 5831 2599 1158 516 230 102 45 20 9 4 1
Time: 0.718084ms | Operations: 176420 | Wins: 2

Promising Observations

On the bright side, there are some promising observations – at this point it's too soon to rule out the possibility of a mere coincidence, but there is a potential for ground-breaking results:

1. It seems like there is a huge underexplored gap (nomen omen) in the work of previous researchers, where their sequences were evaluated on similar, common, “clean” sorting ranges (like 1000, 10K)
2. There is a particular 4-elements sequence ending that is not part of any sequence used as a base population, and yet it emerged in more than half of tested sorting ranges.

Genetic Algorithm v3: Most Common Endings - Last 4 Elements (from 1)



What's Next? – Course of Research

- **GA_{v4}** – I have a few ideas on how to upgrade the current GA solution, possibly with both faster exploration and better exploitation.
- **Faster architecture** – Moving main evaluation part of algorithm to GPU, and utilizing CPU resources freed this way in other areas:
 - more complex population generation,
 - searching simultaneously for different sorting ranges.
- **MORE DATA!** – Running the searching pipeline on departmental resources to gather much more data for further analysis.
- **Results** – Grand analysis of gathered data, drawing conclusions, and presentation of my own gap sequence.

And of course there is still half of Master Thesis to be written :(

Bibliography

- ❖ Ciura, M. (2001). Best increments for the average case of Shellsort. In R. Freivalds (Ed.), *Fundamentals of Computation Theory* (pp. 106–117). Berlin, Germany: Springer.
- ❖ Lee, Y. W. (2021). Empirically improved Tokuda gap sequence in Shellsort. arXiv preprint arXiv:2112.12450.
- ❖ Lee, Y. W. (2021). Optimal gap sequences in Shellsort for $n \leq 16$ elements. arXiv preprint arXiv:2104.05389.
- ❖ Shell, D. L. (1959). A high-speed sorting procedure. *Communications of the ACM*, 2(7), 30–32.
- ❖ Skean, O., Ehrenborg, R., & Jaromczyk, J. W. (2021). *Optimization perspectives on Shellsort*. Lexington, KY: University of Kentucky.
- ❖ Tokuda, N. (1992). An improved Shellsort. In *Proceedings of the IFIP 12th World Computer Congress on Algorithms, Software, Architecture-Information Processing* (Vol. 1, pp. 449–457).

- ❖ And tons of my own knowledge, interpretations and ideas – results will show if they were correct :)

The End

Thank you for your attention.



Presentation by inż. Adam Cherek, s207250